CSC232- INFORMATION SECURITY

Assignment 2: DES and AES

Submitted By:
Muhammad Rizwan Shafiq
SP24-BCS-069
BCS-4B

Instructor:
Sir Sabghat Ullah Khan

Assignment 2: Modern Cryptography (DES and AES)

Introduction

This assignment focuses on modern cryptographic algorithms and their implementation in Python. It extends classical cryptography concepts by introducing AES and DES in different modes, along with hybrid techniques and error propagation analysis. The assignment includes five tasks covering AES, DES, file encryption, and block-level behavior comparisons.

Task 1 – AES with Classical Pre-Processing

Question: Apply a Caesar cipher (shift of 3) before encrypting with AES-CBC. Then decrypt and verify correctness.

This task combines classical and modern cryptography. A Caesar cipher with shift=3 is first applied to the plaintext, and the resulting text is then encrypted using AES in CBC mode. PKCS7 padding ensures block alignment. After decryption, a reverse Caesar shift restores the original text.

Sample Code Snippet:

```
def caesar_shift(text, shift=3):
    res = []
    for ch in text:
        if 'a' <= ch <= 'z': res.append(chr((ord(ch)-97+shift)%26+97))
        elif 'A' <= ch <= 'Z': res.append(chr((ord(ch)-65+shift)%26+65))
        else: res.append(ch)
    return ''.join(res)
cipher = AES.new(key, AES.MODE_CBC, iv)
```

Output Example:
Plaintext: security
Caesar Output: vhfxulwb
AES Ciphertext (Base64): z1qX6XzKUzE5F4...
Recovered Plaintext: security

Task 2 – DES Double Encryption

Question: Implement Double DES (plaintext → DES(K1) → DES(K2)) and test with K1 ≠ K2 and K1 = K2.

The program performs double encryption and decryption using two DES keys (K1 and K2). If both keys are equal (K1 = K2), the cipher behaves like single DES. This demonstrates that double encryption does not double the security if the keys are the same.

Sample Code Snippet:

```
def double_des_encrypt(k1, k2, data):
    c1 = DES.new(k1, DES.MODE_ECB).encrypt(pkcs7_pad(data, 8))
```

```
    c2 = DES.new(k2, DES.MODE_ECB).encrypt(c1)
    return c2
```

Output Example:
K1 = 4af32b...
K2 = 87cd91...
Ciphertext (Base64): T0xNbW0y...
When K1 = K2 → Output behaves as Single DES

Task 3 – AES vs DES Block Size Analysis

Question: Encrypt the same plaintext using AES (128-bit) and DES (64-bit) in ECB mode and compare block sizes.

The task compares AES and DES encryption outputs. AES uses a 128-bit block while DES uses 64-bit. The script prints both ciphertexts in hexadecimal and highlights AES's stronger structure, larger block size, and resistance to cryptanalytic attacks.

Sample Code Snippet:

- ```
  aes_ecb = AES.new(aes_key, AES.MODE_ECB)
  des_ecb = DES.new(des_key, DES.MODE_ECB)
  ct_aes = aes_ecb.encrypt(pkcs7_pad(text, 16))
  ct_des = des_ecb.encrypt(pkcs7_pad(text, 8))
  ```

Observation:
• AES Ciphertext (hex): a6d8...3f9e
• DES Ciphertext (hex): 2b19...9a10
AES provides better diffusion and lower risk of pattern repetition in ECB mode.

Task 4 – File Fragment Encryption with AES and DES

Question: Split a text file into two halves, encrypt the first half with AES-CBC and the second half with DES-ECB, then decrypt and reconstruct the file.

The script automatically reads 'input.txt', encrypts its first half using AES (CBC) and second half using DES (ECB), and combines ciphertexts into 'encrypted.bin'. Metadata such as keys, IV, and lengths are stored in a JSON file to aid decryption.

Sample Code Snippet:

- ```
  aes = AES.new(aes_key, AES.MODE_CBC, aes_iv)
  first_ct = aes.encrypt(pkcs7_pad(first, 16))
  des = DES.new(des_key, DES.MODE_ECB)
  second_ct = des.encrypt(pkcs7_pad(second, 8))
  ```

Output Example:
[+] Encrypted → encrypted.bin

[+] Decrypted → recovered.txt
File successfully recovered with same length and content.

Task 5 – Bit-Flipping Experiment (AES-CBC vs DES-CBC)

Question: Encrypt a short message with AES-CBC and DES-CBC, flip 1 bit in ciphertext, and compare plaintext changes.

This experiment demonstrates error propagation in CBC mode. Flipping one bit in ciphertext corrupts the entire block and partially affects the next block. AES's 16-byte block size spreads errors over a larger area than DES's 8-byte block.

Sample Code Snippet:

- ct_flipped = flip_one_bit(ct, 0, 0)
  dec_flipped = cipher_cls.new(key, cipher_cls.MODE_CBC, iv=iv).decrypt(ct_flipped)
  print(count_byte_diffs(dec_simple, dec_flipped))

Observation:
AES-CBC: Large corruption across 16-byte block.
DES-CBC: Corruption limited to 8-byte block.
Conclusion: Larger block size → higher error diffusion.

Summary

This assignment explored advanced cryptographic operations, bridging classical and modern encryption methods. Students implemented AES and DES in multiple modes, compared security properties, and visualized error propagation. The exercises provided hands-on experience with block ciphers, padding, and hybrid encryption models.